orderings, but these orderings are restricted. The following listing is a rework of listing 5.4 using acquire-release semantics rather than sequentially consistent ones.

Listing 5.7. Acquire-release doesn't imply a total ordering

```
#include <atomic>
#include <thread>
#include <assert.h>

std::atomic<bool> x,y;
std::atomic<int> z;

void write_x()
{
    x.store(true,std::memory_order_release);
}

void write_y()
{
    y.store(true,std::memory_order_release);
}

void read_x_then_y()
{
    while(!x.load(std::memory_order_acquire));
    if(y.load(std::memory_order_acquire))       <-(1)
        ++z;
}

void read_y_then_x()
{
    while(!y.load(std::memory_order_acquire));
    if(x.load(std::memory_order_acquire))       <-(2)
        ++z;
}

int main()
{
    x=false;
    y=false;
    z=0;
    std::thread a(write_x);
    std::thread b(write_y);
    std::thread c(read_x_then_y);
    std::thread d(read_y_then_x);
    a.join();
    b.join();
    c.join();
    d.join();
    assert(z.load()!=0);    <-(3)
}
```

In this case the assert (3) *can* fire (just like in the relaxed-ordering case), because it's possible for both the load of `x` (2) and the load of `y` (1) to read `false`. `x` and `y` are writtenby different threads, so the ordering from the release to the acquire in each case has no effect on the operations in the other threads.